Operating System Orange Problem

Name: Gaurav Girish

SRN: PES1UG24AM103

Section: AIML B

Development Outputs

Output 1a: Phase 1 - Object Storage Unit Tests

```
gaurav@gaurav-QEMU-Virtual-Machine:~/PES1UG24AM103-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Output 1b: Phase 1 - Verified Object Directory Structure

```
gaurav@gaurav-QEMU-Virtual-Machine:~/PES1UG24AM103-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
gaurav@gaurav-QEMU-Virtual-Machine:~/PES1UG24AM103-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

Output 2a: Phase 2 - Binary Representation of Tree Object

```
gaurav@gaurav-QEMU-Virtual-Machine:~/PES1UG24AM103-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Output 2b: Phase 2 - Tree Serialization and Parsing Tests

```
gaurav@gaurav-QEMU-Virtual-Machine:~/PES1UG24AM103-pes-vcs$ xxd .pes/objects/37/1aac45c3ba401d13c2ce98c9537b4e53c6049b95
0bf8a33cbc400712d9f4fa | head -20
00000000: 7472 6565 2034 3500 3130 3036 3434 2061  tree 45.100644 a
00000010: 2e74 7874 002c f8d8 3d9e e295 43b3 4a87  .txt.,..=...C.J.
00000020: 7274 21fd ecb7 e3f3 a183 d337 6390 25de  rt!........7c.%.
00000030: 576d b9eb b4                             Wm...
```

Output 3a: Phase 3 - Initialization, Staging, and Status Command

```
gaurav@gaurav-QEMU-Virtual-Machine:~/PES1UG24AM103-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
gaurav@gaurav-QEMU-Virtual-Machine:~/PES1UG24AM103-pes-vcs$ echo "hello" > file1.txt
gaurav@gaurav-QEMU-Virtual-Machine:~/PES1UG24AM103-pes-vcs$ echo "world" > file2.txt
gaurav@gaurav-QEMU-Virtual-Machine:~/PES1UG24AM103-pes-vcs$ ./pes add file1.txt file2.txt
added: file1.txt
added: file2.txt
gaurav@gaurav-QEMU-Virtual-Machine:~/PES1UG24AM103-pes-vcs$ ./pes status
Staged changes:
    staged: file1.txt
    staged: file2.txt
gaurav@gaurav-QEMU-Virtual-Machine:~/PES1UG24AM103-pes-vcs$
```

Output 3b: Phase 3 - Internal Index File Inspection

```
gaurav@gaurav-QEMU-Virtual-Machine:~/PES1UG24AM103-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776777358 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35ff7e65d6f606199f7f 1776777366 6 file2.txt
```

Output 4a: Phase 4 - Repository File Hierarchy

```
gaurav@gaurav-QEMU-Virtual-Machine:~/PES1UG24AM103-pes-vcs$ ./pes log
commit a9a49baf53a199a3983b42851218c4646409a9b534495bae55ca0eead589e2d7
Author: gaurav
Date:   1776777686

    Add farewell

commit befda4d94cf4e32e458722a1d8bd4c0afa39647bdd5bf790ea4ef53f6974b407
Author: gaurav
Date:   1776777649

    Add world

commit f1f055c5823a531e6caf1574f2fe3256803297774be1bf423fe7bb03d96c791a
Author: gaurav
Date:   1776777616

    Initial commit◆n◆◆

gaurav@gaurav-QEMU-Virtual-Machine:~/PES1UG24AM103-pes-vcs$
```

Output 4b: Phase 4 - Branch Reference and HEAD Verification

```
gaurav@gaurav-QEMU-Virtual-Machine:~/PES1UG24AM103-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5[illegible]af7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/28/7b97e0e3ae3938b05cda855c75bd7619b31f2f852993b2fdfb02ba3ac00c95
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/a9/a49baf53a199a3983b42851218c4646409a9b534495bae55ca0eead589e2d7
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/be/fda4d94cf4e32e458722a1d8bd4c0afa39647bdd5bf790ea4ef53f6974b407
.pes/objects/c1/3c9de65743a09cccaab64a44c7cf6ed462d9bd5efad5bb8c93865027b1edf6
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f1/f055c5823a531e6caf1574f2fe3256803297774be1bf423fe7bb03d96c791a
.pes/refs/heads/main
```

Output 4c: Phase 4 - Repository Commit History (Log)

```
gaurav@gaurav-QEMU-Virtual-Machine:~/PES1UG24AM103-pes-vcs$ cat .pes/refs/heads/main
a9a49baf53a199a3983b42851218c4646409a9b534495bae55ca0eead589e2d7
gaurav@gaurav-QEMU-Virtual-Machine:~/PES1UG24AM103-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Phase 5 & 6 Assessment Questions

Phase 5: Branching and Checkout

Q5.1 — How would you implement `pes checkout <branch>`?

A branch is just a file in `.pes/refs/heads/` containing a commit hash. To implement checkout, you would: (1) Read the target branch file to get its commit hash. (2) From that commit, read the tree object. (3) Recursively walk the tree, recreating all files and directories in the working directory to match the snapshot. (4) Update `.pes/HEAD` to contain `ref: refs/heads/<branch>`. The complexity comes from step 3 — you must delete files that exist in the current branch but not in the target, and create files that exist in the target but not currently. This requires comparing two trees, not just copying one.

Q5.2 — How do you detect a "dirty working directory" conflict?

For each file in the current index, stat it in the working directory and compare mtime and size to the index entry. If they differ, the file is "dirty" (modified but not staged). Then compare the current branch's committed tree to the target branch's committed tree — if a dirty file differs between the two, refuse the checkout because we would overwrite unsaved changes. This can be done entirely using the index (for staged state), working directory stats (for unstaged changes), and the two commit trees (for branch differences).

Q5.3 — What is "detached HEAD" and how to recover?

Detached HEAD means `.pes/HEAD` contains a commit hash directly instead of `ref: refs/heads/main`. If you make commits in this state, the commits are created normally, but no branch pointer is updated to track them. To recover: note the commit hash printed after each commit, create a new branch pointing to the last commit hash with `git branch recovery-branch <hash>`, then checkout that branch. This "reattaches" the commits to a named branch so they won't be lost to garbage collection.

Phase 6: Garbage Collection

Q6.1 — Algorithm to find and delete unreachable objects:

Start from all branch refs (read every file in `.pes/refs/heads/`). From each branch, walk the commit chain following `parent` pointers until there's no parent. For each commit visited, add its tree hash to a "reachable" set. Recursively traverse each tree: for each entry, if it's a blob add its hash to the reachable set; if it's a subtree, recurse. The data structure is a hash set (e.g., a C hash table or even a sorted array of `ObjectID`). After traversal, scan every file under `.pes/objects/`. For each file, compute its `ObjectID` from the path and check if it exists in the reachable set — if not, delete it. For 100,000 commits with an average of 10 objects per commit, you'd visit roughly 1 million objects total.

Q6.2 — Race condition between GC and concurrent commit:

The race: Thread A (GC) scans the object store and determines blob X is unreachable. Thread B (commit) starts, calls `object_write` to store a new blob X as part of a tree. Thread A deletes blob X before Thread B's commit finishes writing the tree and commit objects that would reference X. When Thread B completes, the commit points to a tree that points to a now-deleted blob — repository corruption. Git avoids this by: (1) using a "grace period" — GC never deletes objects newer than 2 weeks old, since a concurrent writer would have to finish within that window; (2) writing objects before writing refs, so GC can never see a ref pointing to something that doesn't exist yet.